Discover : répartition du backend

Suite de `repartition.md`, qui répartit le frontend Flutter et décrit les spécifications de l'API (routes, règles communes, règles métier). Ce document répartit l'écriture de cette API entre Siméon, Joël et Djamal.

Le backend suit la répartition du frontend : chacun écrit les routes que ses écrans appellent. Les écrans d'Ainé Mao sont repris ainsi : le profil va à Siméon, qui gère déjà les comptes, et la publication est partagée entre Siméon (le téléversement) et Djamal (les contenus).

Membre	Partie	Routes	Charge
Siméon	Socle, compte, profil, téléversement	`auth/*`, `moi/*`, `envois/*`, `modeles-telephone`	Socle + 11 points
Joël	Exploration, lieux, créateurs, notifications	`flux/*`, `carte`, `recherche`, `lieux/*`, `utilisateurs/*`, `notifications/*`	13 points
Djamal	Contenus, reels, communauté, publication	`contenus/*`, `reels`	12 points

Même échelle : S = 1 point, M = 2 points, L = 4 points.

Siméon : socle, compte, profil et téléversement

Le socle, utilisé par tout le monde, à pousser en premier : projet Laravel dans `discover_api/`, Sanctum, préfixe `/api/v1`, format d'erreur `{ message, erreurs }`, **toutes les migrations, modèles, fabriques et données de démonstration** (utilisateurs, lieux, contenus, envois, réactions, commentaires, abonnements, notifications, modèles de téléphone), stockage S3, base des tests.

Domaine	Routes	Écrans	Taille
Compte	`POST auth/email`, `auth/inscription`, `auth/code`, `auth/code/renvoyer`, `auth/connexion`, `auth/google`, `auth/oubli`, `auth/nouveau-mot-de-passe`, ✓ `auth/deconnexion`	E02	L
Mon profil	✓ `GET moi`, ✓ `PATCH moi`, ✓ `POST moi/avatar`, ✓ `DELETE moi`, ✓ `GET moi/contenus`	E36, E18, E21	M
Téléversement	✓ `POST envois`, ✓ `HEAD envois/{envoi}`, ✓ `PATCH envois/{envoi}`, ✓ `POST envois/{envoi}/terminer`	E12, E16	L
Téléphones	✓ `GET modeles-telephone?q=`, ✓ `POST modeles-telephone`	E15	S

Règles métier à sa charge : `auth/email` ne révèle rien du compte, code à 6 chiffres haché valable 10 minutes, blocage de 15 minutes après 5 essais manqués (429).

Joël : exploration, lieux, créateurs et notifications

Domaine	Routes	Écrans	Taille
Flux	`GET flux/decouverte`, ✓ `GET flux/abonnements`, avec les filtres	E03, E04	M
Près de moi et carte	`GET flux/pres-de-moi`, `GET carte` (mêmes requêtes géographiques)	E03, E11	L
Recherche	`GET recherche?q=`	E31	M
Lieux	`GET lieux/{lieu}`, `GET lieux/{lieu}/contenus`	E33	S
Créateurs et abonnements	`GET utilisateurs/{nom}`, `GET utilisateurs/{nom}/contenus`, ✓ `PUT` et `DELETE utilisateurs/{nom}/abonnement`	E20	M
Notifications	✓ `GET notifications`, ✓ `POST notifications/lues`, et le service qui les crée	E17	M

La carte change de main par rapport au frontend : la spécification la range dans l'exploration et elle partage ses requêtes avec « Près de moi ».

Djamal : contenus, reels, communauté et publication

Domaine	Routes	Écrans	Taille
Fiche d'un contenu	`GET contenus/{contenu}` avec son badge	E05	S
Reels	`GET reels`	E06	M
Réactions	✓ `PUT` et `DELETE contenus/{contenu}/reaction`	E05, E06	S
Commentaires	`GET contenus/{contenu}/commentaires`, ✓ `POST contenus/{contenu}/commentaires`	E07	M
Publication	✓ `POST contenus`, ✓ `PATCH contenus/{contenu}`, ✓ `DELETE contenus/{contenu}`	E14, E15, E30, E16	M
Analyse du fichier	lecture des métadonnées (téléphone, date, lieu), refus des fichiers qui portent des signes d'IA, choix du badge	E15, E16	L

Points de contact entre les parties

- **Analyse du fichier :** Djamal écrit un service `AnalyseurFichier` ; Siméon l'appelle dans `envois/{envoi}/terminer`, qui renvoie ce que le fichier contient déjà, pour que l'application sache quoi faire compléter (E15). On fixe ensemble le format de cette réponse avant de commencer.
- **Notifications :** Joël écrit le service qui crée une notification ; Djamal l'appelle après une réaction ou un commentaire, Joël après un abonnement.
- **Format d'un contenu :** une seule ressource Laravel `ContenuResource`, écrite par Djamal, réutilisée par tous les flux, la carte, la recherche et les profils.
- **Base de données :** seul Siméon ajoute des migrations au début. Une colonne manquante se demande à Siméon, ou s'ajoute dans une nouvelle migration après accord.

Comment travailler sur le backend

1. Siméon pousse le socle sur la branche `simeon-api`. Une fois fusionné dans `main`, chacun part de `main` avec une branche `joel-api` ou `djamal-api`.
2. Chacun écrit les tests de ses routes ; `php artisan test` doit passer avant la demande de fusion.
3. Chacun remplace ensuite, côté Flutter, ses données simulées par une version API, sur le modèle de `fonctionnalites/compte/donnees/depot_compte_api.dart`. Le profil et la publication passent à l'API avec Ainé Mao.
4. Aucun secret dans le dépôt : `.env` reste local, seul `.env.example` est versionné.